

Veri Dönüştürme / Ölçeklendirme & Hazırlama

Veri Dönüşümüne Neden İhtiyaç Duyulur (Sayısal Veriler İçin)

- Veri setlerinde değişkenler genelde farklı birimlerde veya ölçeklerde olabilir — örneğin “yaş (yıl)”, “gelir (TL)”, “metrekare (m²)”, “oda sayısı (1–5)” gibi. Eğer bu değişkenler direkt modele verilirse, büyük değer aralıklarına sahip değişkenler modelin yanılmasına sebep olabilir(daha önemli olduğu noktasında). — Bu yüzden değişkenleri aynı ölçeğe getirmek gerekir.
- Bazı algoritmalar özellikle bu dönüşümleri gerektirir:
 - Mesafe/benzerlik temelli algoritmalar (örneğin k-NN, k-means, SVM, vb), çünkü bu algoritmalar değişkenler arasındaki mutlak büyüklük farkını kullanır.
 - Gradient-descent ile öğrenen modeller (lineer/lojistik regresyon, neural network gibi) — Dönüşüm, optimizasyonun daha hızlı ve stabil çalışmasına yardımcı olur.
- Dönüşüm bazı durumlarda sayısal kararlılığı, sayısal sapmaları, outlier etkisini azaltır; modelin genel başarısını artırır.
- Ancak “her zaman dönüşüm yapmak zorunludur” gibi bir kural yoktur — Veri yapısı, algoritma, değişken tipi, outlier varlığı vs’ye göre karar vermek gerekir.

Yaygın Veri Dönüştürme / Özellik Ölçeklendirme Teknikleri

Teknik Adı	Açıklama / Ne Yapar	Ne Zaman Kullanılır / Avantajı
Standardizasyon (Z-score)	Her bir değişken için: $((x - \mu) / \sigma) \rightarrow$ ortalaması 0, standart sapması 1 olacak şekilde dönüştürür.	Özellikle değişkenler farklı birim/ölçeklerdeyken; normal dağılım varsayımı olan ya da gradient-bazlı algoritmalar için iyi.
Min-Max Normalizasyon (0–1 aralığına ölçekleme)	Her bir değişkeni en düşük değer 0, en yüksek değer 1 olacak biçimde sıkıştırır. $(x' = (x - x_{\min}) / (x_{\max} - x_{\min}))$	Özellikle değişkenler çok farklı aralıklarda; ve outlier yok ya da az ise; görselleştirme ya da bazı algoritmalar için.
Robust Scaling	Ortalama/standart sapma yerine, medyan ve çeyreklik aralığı (IQR) kullanır; bu sayede outlier'lara karşı daha dayanıklı olur.	Veri setinde outlier varsa ve bu outlier'ların etkisi azaltılmak isteniyorsa

```
In [3]: # veri seti oluşturma

import pandas as pd
import numpy as np

# Örnek veri
np.random.seed(0)
df = pd.DataFrame({
    "yas": np.random.randint(18,80, size=100),          # yaş (yıl)
```

```
"gelir": np.random.normal(50000, 15000, size=100), # gelir
"evbuyuklugu": np.random.normal(120, 30, size=100), # metrekaresi
"odasayisi": np.random.randint(1,6, size=100) # oda sayısı
})

print(df.head())
df.describe().T
```

```
   yas      gelir  evbuyuklugu  odasayisi
0   62  56080.966760      76.852744         1
1   65  51779.098779     128.137465         1
2   71  68816.211025     99.281906         3
3   18  71286.530612     99.554011         5
4   21  38842.158757     74.633081         4
```

```
Out[3]:
```

		count	mean	std	min	25%	75%
	yas	100.0	46.620000	18.117897	18.000000	31.750000	46.500000
	gelir	100.0	49336.921602	15665.139033	12238.443458	40857.598263	49537.200000
	evbuyuklugu	100.0	119.911821	29.765846	33.666638	100.136907	118.600000
	odasayisi	100.0	3.130000	1.368144	1.000000	2.000000	3.000000

```
In [6]: # Standardizasyon (Z-score) ile Ölçeklendirme
# Bu işlemle tüm değişkenlerin ortalaması ≈ 0, standart sapması ≈ 1 olur.

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_std = df.copy()
df_std[df.columns] = scaler.fit_transform(df)

df_std.describe().T
```

```
Out[6]:
```

		count	mean	std	min	25%	50%	75%
	yas	100.0	1.237899e-16	1.005038	-1.587612	-0.824870	-0.006657	0.769953
	gelir	100.0	-1.287859e-16	1.005038	-2.380150	-0.544013	0.012851	0.825164
	evbuyuklugu	100.0	-4.940492e-16	1.005038	-2.912051	-0.667696	-0.043342	0.751259
	odasayisi	100.0	1.088019e-16	1.005038	-1.564697	-0.830097	-0.095498	0.639101

```
In [7]: # Min-Max Normalizasyona Göre Ölçeklendirme
# Tüm öznitelikler [0, 1] aralığındadır.

from sklearn.preprocessing import MinMaxScaler

scaler_mm = MinMaxScaler()
df_mm = df.copy()
df_mm[df.columns] = scaler_mm.fit_transform(df)
```

```
df_mm.describe().T
```

Out[7]:

	count	mean	std	min	25%	50%	75%	max
yas	100.0	0.485085	0.307083	0.0	0.233051	0.483051	0.720339	1.0
gelir	100.0	0.558809	0.235962	0.0	0.431086	0.561826	0.752540	1.0
evbuyuklugu	100.0	0.566508	0.195519	0.0	0.436615	0.558076	0.712657	1.0
odasayisi	100.0	0.532500	0.342036	0.0	0.250000	0.500000	0.750000	1.0

In [8]:

```
# Robust Scaling
# Eğer veri setinizde uç değerler / outlier varsa
# bu yöntem, standart sapma / ortalama yerine medyan / IQR kullandığı içi

from sklearn.preprocessing import RobustScaler

scaler_rb = RobustScaler()
df_rb = df.copy()
df_rb[df.columns] = scaler_rb.fit_transform(df)

df_rb.describe().T
```

Out[8]:

	count	mean	std	min	25%	50%	75%
yas	100.0	0.004174	0.630188	-0.991304	-0.513043	0.000000e+00	0.486957
gelir	100.0	-0.009386	0.734045	-1.747766	-0.406715	0.000000e+00	0.593285
evbuyuklugu	100.0	0.030545	0.708294	-2.021706	-0.440010	1.682682e-16	0.559990
odasayisi	100.0	0.065000	0.684072	-1.000000	-0.500000	0.000000e+00	0.500000



Kategorik Değişken Kodlama (One-Hot, LabelEncoding, get_dummies)

Neden “Kodlama / Encoding” Gereklidir?

- Makine öğrenmesi algoritmalarının (çoğu) yalnızca sayısal (numeric) girdilerle çalışır. Eğer veri setinde metin/kategorik değişken varsa — “Kırmızı / Mavi / Yeşil”, “Erkek / Kadın”, “A / B / C şehirleri” gibi — bunlar doğrudan modele verilemez.
- Bu nedenle kategorik değişkenlerin sayısal temsillere dönüştürülmesi gerekir. Bu sürece “encoding / kodlama” adı verilir.
- Ancak kodlama yapılırken dikkat edilmelidir. Kodlama şekli, modelin bu değişkenleri nasıl yorumlayacağı açısından önemlidir — yanlış kodlama, modele “gereksiz sıralama / ilişki / anlam” yükleyebilir.

Temel Yöntemler: Label Encoding vs One-Hot vs get_dummies

Label Encoding

- Her farklı kategoriye bir tam sayı (integer) atar. Örneğin ["Apple", "Banana", "Orange"] kategorisini 0, 1, 2 gibi sayılara dönüştürür.

- Uygun olduğu durum: Eğer kategorik değişken ordinal ise — yani kategoriler arasında doğal bir sıralama/öncelik ilişkisi varsa (örneğin: “düşük – orta – yüksek”, “küçük – orta – büyük”).
- Dezavantaj: Eğer kategoriler nominal (sırasız) ise — yani “renk”, “şehir”, “ürün türü” gibi bir sıralama yoksa — Label Encoding modele yanlışlıkla “sıralı / aralıklı / üstünlük” algısı kazandırabilir. Bu da modelde hatalı çıkarımlara yol açar.

One-Hot Encoding

- Her kategori için ayrı bir binary (0/1) sütun oluşturur. Eğer bir gözlem o kategoriye aitse 1, değilse 0 atar.
- Uygun olduğu durum: Kategorik değişken nominal (sırasız) ise; modelin kategori arasına “sıralama / büyüklük” yüklemesi istenmediğinde kullanılır.
- Dezavantaj: Eğer bir değişkende çok sayıda benzersiz kategori varsa → sütun sayısı (feature sayısı) çok artar; bu “boyut artışı (curse of dimensionality)” ve “sparse veri” problemlerine yol açabilir.

Curse of Dimensionality

- Boyut” = modelde ya da veri setinde yer alan özellik (feature / değişken) sayısı. Boyut arttıkça, veri uzayının geometrisi büyük biçimde değişir.
- Curse of Dimensionality, yüksek boyutlu veri ile ortaya çıkan bir dizi sorun / zorluğu topluca tanımlamaktadır:
 - **Veri seyrekleşir (sparsity / data sparsity):** Özellikle gözlem sayısına göre boyut çok artmışsa — veriler, yüksek boyutlu uzayda birbirinden çok uzak düşer — veri noktaları arası uzaklık artar, “benzerlik / komşuluk” anlamı zayıflar.
 - **Mesafe / uzaklık & benzerlik ölçümleri anlamsızlaşır:** Örneğin Euclidean ya da diğer mesafe temelli algoritmalar — yüksek boyutlarda “en yakın komşu” kavramı etkisizleşir çünkü tüm noktalar neredeyse eşit uzaklıktadır.
 - **Aşırı model karmaşıklığı & Overfitting riski:** Boyut arttıkça model “zorunlu” olmayan, gürültü ya da aleatory (rastgelelik) içeren ilişkileri de öğrenebilir — bu da modelin genelleme (genel veri üzerinde) başarısını düşürür.
 - **Hesaplama / Bellek / Zaman Maliyeti:** Daha fazla boyut → daha çok hesaplama, daha fazla bellek, daha karmaşık optimizasyon → pratik olarak büyük veri setlerinde yük anlamına gelir.
 - **Veri Görselleştirme ve Anlamlandırma Zorluğu:** İnsan zihni 3 boyuta kadar görselleştirmede rahattır; çok boyutlu veri setlerinde desenleri, ilişkileri anlamak zorlaşır.
- **Özetle:** Özellik sayısı (dimensiyon) arttıkça, “her şeyi bilmeye” çalışma cazip görünse de — pratikte hem veri ihtiyacı, hem model karmaşıklığı, hem hata / genelleme riskleri artar.

Sparse / Seyrek Veri Problemi

- Sparse (seyrek) veri, bir veri kümesinde pek çok özellik / değişken olmasına rağmen, her gözlemde yalnızca birkaç özelliğin aktif / dolu / anlamlı olduğu durumları tanımlar — bu özellikle “yüksek boyutlu / geniş” veri setlerinde yaygındır (örneğin metin verisi, one-hot encoded kategorik veriler, kullanıcı-item matrisleri, vs.).

- Yüksek boyut + az ilgili veri → seyrek matris (çoğu hücre “0” ya da eksik / anlamsız).
Bu durumda:
 - Mesafe / benzerlik temelli modeller (örneğin KNN, K-means) kadar başarılı çalışmayabilir: çünkü “benzerlik” tanımı bulanıklaşır, komşu bulmak zorlaşır
 - İstatistiksel yöntemlerin / modelin öğrenebileceği anlamlı desen sayısı azalır — çünkü veri “bilgi” yerine “noise / boşluk” içeriyor olabilir.
 - Hesaplama ve bellek maliyeti artış + verimli olmayan veri temsili (çok sayıda 0, gereksiz sütunlar)

Curse of Dimensionality ile sparse veri problemi genelde birlikte anılır — yüksek boyut, verinin seyrekleşmesine yol açar.

Bu Problemler Neden Tehlikeli? — Hangi Alanlarda Etki Gösterir?

- **Mesafe / benzerlik temelli algoritmalar:** K-Nearest Neighbors (KNN), K-means, kümeleme, yoğunluk tabanlı outlier detection gibi yöntemler — mesafe & yoğunluk varsayımlarına dayanır. Yüksek boyut + sparse veri → mesafeler anlamsızlaşır, komşu & yoğunluk temelli değerlendirme bozulur.
- **Model genellemesi :** Özellikle küçük veri + çok değişken (özellik) olduğunda — model, eğitim verisine çok uyum (overfitting) yapar; yeni / gerçek veri geldiğinde zayıf performans verir.
- **Özellik gereksizliği / multicollinearity:** Çok sayıda özellik varsa, bazıları gereksiz, redundant ya da birbirleriyle yüksek korelasyondadır — bu da modelin istikrarını, yorumlanabilirliğini bozar.
- **Yüksek hesaplama / zaman / bellek maliyeti:** Özellikle büyük veri setlerinde — eğitim süresi, bellek ihtiyacı, disk & CPU yükü ciddi artar. Bu pratikte hem kaynak tüketimini artırır hem de ölçeklenebilirliği azaltır.

Bu Problemlere Karşı Stratejiler — Nasıl Ele Alınmalı?

Strateji / Yöntem	Ne Yapar / Neden Etkili	Ne Zaman Uygun
Özellik Seçimi (Feature Selection)	Gereksiz / redundant / düşük katkılı değişkenleri çıkarır. Böylece boyut azalır, model daha sade olur, overfitting riski düşer.	Özellik sayısı çok fazla, bazıları bilgi az taşıyorsa; özellikle tabular veri
Boyut İndirgeme (Dimensionality Reduction) — örneğin Principal Component Analysis (PCA), t-SNE, vb.	Orijinal değişkenlerin lineer / non-linear kombinasyonlarıyla daha az sayıda bileşen + varyansı koruyarak yeni alt uzaya indirir. Böylece veri yoğunlaşır, mesafe / benzerlik / modelleme daha stabil olur.	Özellikle sayısal, yüksek boyutlu veri; model karmaşıklığı ve boyutu azaltılmak istendiğinde
Regularization (Düzenleştirme) — L1 / L2 / Ridge / Lasso gibi	Modelin “gereksiz değişkenleri / gürültüyü” öğrenmesini engeller, katsayıları küçültür veya bazı değişkenleri sıfıra iter → böylece overfitting azalır.	Lineer modellerde; yüksek boyut + az veri olduğunda
Seyrek veri temsili / Sparse-aware algoritmalar (sparse matris, feature hashing, embedding, vs.)	Veri belleğini ve hesaplamayı optimize eder; yalnızca aktif / önemli özellikleri saklamaya yönelir.	Özellikle “çok sayıda kategorik değişken”, “one-hot kodlama sonrası geniş sparse matris”, “text / bag-of-words / TF-IDF” gibi durumlarda

Strateji / Yöntem	Ne Yapar / Neden Etkili	Ne Zaman Uygun
Daha fazla veri toplamak / veri arttırımı (data augmentation)	Özellikle yüksek boyutlu veri için — veri noktası ne kadar çok olursa, yüksek boyutun negatif etkisi azalır. Ancak pratikte her zaman mümkün olmayabilir.	Veri toplamak mümkünse; değilse diğer stratejilerle birlikte
Algoritma seçimi	Bazı algoritmalar yüksek boyutlu & sparse veriye daha dayanıklıdır (örneğin ağaç tabanlı modeller, regularize edilmiş modeller, embedding-temelli modeller, vb.)	Yüksek boyut + sparse veri olduğu durumlarda — özellikle KNN / k-means yerine alternatif modeller

pandas.get_dummies

- One-Hot Encoding'in hızlı ve pratik bir yoludur — pandas DataFrame için doğrudan binary sütunlar oluşturur.
- Örneğin `df = pd.get_dummies(df, columns=['Color'])` ile Color sütunundaki "Red, Green, Blue" kategorileri → Color_Red, Color_Green, Color_Blue sütunlarına dönüştürülür.
- Eğer `drop_first=True` parametresi verilirse, ilk kategori için sütun atlanır — bu, "dummy variable trap" (çoklu doğrusal bağ/ multicollinearity) riskini azaltır.

Hangi Yöntemi Ne Zaman Kullanmalı?

Durum / Veri Özelliği	Tavsiye Edilen Kodlama
Kategorik değişken nominal (sırasız) — örn. renk, cinsiyet, şehir, ürün türü vs	One-Hot Encoding / get_dummies
Kategorik değişken ordinal (sıralı) — örn. "küçük-orta-büyük", "düşük-orta-yüksek", eğitim seviyesi gibi	Label Encoding (veya ordinal encoding)
Az sayıda kategori & model "boyut artışı" problemi değilse	One-Hot / get_dummies
Çok fazla benzersiz kategori varsa → yüksek boyut yaratacaksa	Label Encoding, veya alternatif kodlama / embedding / target encoding vb (ancak bu özel, dikkatli düşünülmeli)

```
In [13]: # veri aolusturma

import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Örnek veri
df = pd.DataFrame({
    "Color": ["Red", "Green", "Blue", "Green", "Red", "Blue"],
    "Size": ["S", "M", "L", "XL", "M", "S"],
    "Price": [10.5, 12.0, 9.5, 14.0, 11.0, 10.0]
})

df
```

Out[13]:

	Color	Size	Price
0	Red	S	10.5
1	Green	M	12.0
2	Blue	L	9.5
3	Green	XL	14.0
4	Red	M	11.0
5	Blue	S	10.0

```
In [14]: # Label Encoding

le = LabelEncoder()
df_le = df.copy()
df_le["Color_enc"] = le.fit_transform(df_le["Color"])
df_le["Size_enc"] = le.fit_transform(df_le["Size"])
print("\nLabel Encoded DF:")
df_le
```

Label Encoded DF:

Out[14]:

	Color	Size	Price	Color_enc	Size_enc
0	Red	S	10.5	2	2
1	Green	M	12.0	1	1
2	Blue	L	9.5	0	0
3	Green	XL	14.0	1	3
4	Red	M	11.0	2	1
5	Blue	S	10.0	0	2

```
In [15]: # One-Hot Encoding ile get_dummies (pandas)

df_ohe = pd.get_dummies(df, columns=["Color", "Size"], drop_first=False,
df_ohe
```

Out[15]:

	Price	Color_Blue	Color_Green	Color_Red	Size_L	Size_M	Size_S	Size_XL
0	10.5	0	0	1	0	0	1	0
1	12.0	0	1	0	0	1	0	0
2	9.5	1	0	0	1	0	0	0
3	14.0	0	1	0	0	0	0	1
4	11.0	0	0	1	0	1	0	0
5	10.0	1	0	0	0	0	1	0

```
In [16]: # One-Hot / sklearn OneHotEncoder + Pipeline

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor # ya da başka
```

```

categorical_features = ["Color", "Size"]
numeric_features = ["Price"]

preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(drop=None, sparse_output=False), categorical_features),
        # numeric özellikler olduğu gibi geçebilir
    ],
    remainder="passthrough"
)

pipe = Pipeline(steps=[
    ("preproc", preprocessor),
    ("model", RandomForestRegressor())
])

X = df.drop("Price", axis=1)
y = df["Price"]
pipe.fit(X, y)
print(pipe.predict(X.head()))

```

```
[10.63  12.075 10.03  13.105 11.005]
```

ColumnTransformer

- ColumnTransformer, bir veri setindeki farklı sütunlara (özelliklere) farklı ön işleme (preprocessing / transformation) uygulamayı sağlayan sınıftır.
- Örneğin: bazı sütunlar kategorik (ör. "renk", "şehir"), bazıları sayısal (ör. "yaş", "gelir") olabilir. Kategorik olanlara encoding (One-Hot etc.), sayısal olanlara scaling / standardizasyon / başka dönüşümler uygulamak gerekebilir. ColumnTransformer, hepsini tek kod bloğunda düzenli bir şekilde tanımlamayı sağlar.
- Bu yaklaşım, preprocessing adımını sadeleştirir, okunabilirliği artırır ve hataları azaltır.

transformers=[...]

- Bu parametre bir liste alır; her eleman bir üçlü ((isim, transformer, sütun listesi)) olur. "cat" → bu sizin verdiğiniz isimdir.
- OneHotEncoder(drop=None, sparse_output=False) → bu, hangi ön işlemin uygulanacağını tanımlar:
- categorical_features → değişken adı ya da sütun listesi; bu encoding'in uygulanacağı sütun(lar).

Transformer Parametreleri

- OneHotEncoder → Kategorik değişkenleri "one-hot / dummy" sütunlarına çeviren çeviricidir. Her kategori için ayrı bir 0/1 sütunu oluşturur.
- drop=None → Bu parametre, One-Hot sırasında bir kategorinin ("first" vs) düşürülüp düşürülmeyeceğini belirler. None demek tüm kategoriler için sütun oluştur — yani "dummy variable trap" vs düşürülmüyorsa tüm sütunlar tutulur.
- sparse_output=False → Default olarak, OneHotEncoder çıktı olarak sparse (seyrek) matris döndürür — çok sayıda sütun ve genelde çoğu 0 olacağı için. Ama

`sparse_output=False` verildiğinde, çıktı dense (yoğun / numpy array / normal matris) olur. Bu sayede genelde DataFrame ya da numpy array ile işlem yapmak kolaylaşır.

`remainder="passthrough"`

- `remainder` parametresi — transformer listesinde yer almayan (yani özel işlem belirtilmemiş) sütunlarla ilgili ne yapılacağını belirler.
- `"passthrough"` demek: “transformers ile özel bir dönüşüm uygulanmamış olan sütunları, oldukları gibi bırak (ön işlem uygulama), ve çıktı sütunlarına dahil et.” demektir.
- Eğer bunu yazılmadıysa, varsayılan `"drop"` olur — yani özel işlem tanımlanmayan sütunlar çıktıdan çıkarılır.

Not: Eğer kategorik sütun çok sayıda kategori içeriyorsa ve one-hot sonrası sütun sayısı çok artacaksa, sparse matris kullanmak bellek açısından avantajlı olabilir. Fakat bazen dense çıktı istenildiği için `sparse_output=False` kullanılır.

In []: